This architecture is designed for high availability, consistency, and regulatory compliance, utilizing a **Microservices-oriented modular monolith** approach to balance development velocity with system performance.

---

1. Architecture Overview

- **Pattern:** Modular Monolith (starting) with a pathway to Microservices.
- **API Gateway:** Kong or NGINX to handle rate limiting, request routing, and central authentication validation.
- **Database:** PostgreSQL (Primary), Redis (Caching/Session), Elasticsearch (Audit/Log searching).
- **Event Bus:** RabbitMQ or Apache Kafka for asynchronous processing (Fraud Detection, Notifications).

2. Services

- **Identity Service:** User management, MFA, RBAC.
- **Account Service:** Ledger operations, balance management, internal transfers.
- **Loan Service:** Workflow state machine, application processing.
- **Analytics/AI Service:** Python-based worker consuming transaction streams for insights.
- **Fraud Service:** Rule-based engine + Async validation checks.

3. Modules (Internal logic groupings)

- **Ledger Module:** Handles double-entry bookkeeping logic.
- **Notification Module:** Multi-channel (Email, SMS) for MFA and alerts.
- **Report Module:** Aggregates data for regulatory compliance.

4. Authentication & Authorization

- **Auth Flow:** OpenID Connect (OIDC) via OAuth2.
- **MFA:** TOTP (Google Authenticator) or WebAuthn.
- **Authorization:** Attribute-Based Access Control (ABAC) to handle complex roles (e.g., "Employee can only approve loans for their assigned branch").

5. Business Logic

- **Transactions:** Implement the **Saga Pattern** (Orchestration) to handle distributed transactions if moving to microservices.
- **Loan Logic:** Encapsulated in a State Machine (Submitted -> In-Review -> Approved/Denied).
- **Fraud Logic:** Interceptor pattern; every transaction request passes through a `FraudDetectionInterceptor`.

6. Background Jobs

- **Tool:** BullMQ (Node.js) or Celery (Python).
- **Task Types:**

* End-of-day reconciliation.

* Daily AI Insight generation (batch processing).

* Transactional email dispatch.

7. File Storage

- **Object Storage:** AWS S3 or MinIO.
- **Usage:** Encrypted storage for scanned ID documents/loan support docs. Use pre-signed URLs for secure, time-limited access.

8. Error Handling

- **Global Exception Handler:** Centralized mapping of custom application exceptions to HTTP status codes.
- **Circuit Breaker:** Use Hystrix or Resilience4j patterns to handle external API failures (e.g., KYC provider down).

9. Logging & Monitoring

- **Distributed Tracing:** OpenTelemetry for tracing transactions across services.
- **Centralized Logging:** ELK Stack (Elasticsearch, Logstash, Kibana).
- **Alerting:** Prometheus + Grafana for business metrics (e.g., balance integrity, transaction latency).

10. Folder Structure (Clean Architecture)

```
/src
  /modules
    /accounts
      /domain     (Entities)
      /use-cases  (Business logic)
      /infra      (Repositories/DB)
      /interfaces (Controllers)
    /loans
    /fraud
  /shared
    /auth         (Middleware, Guards)
    /logger
    /events       (Event bus emitters)
  /config         (Env setup)
  /tests          (Unit/Integration/E2E)
```

11. Recommended Libraries

- **Backend:** Node.js (NestJS) or Go (Gin/Echo) for high concurrency.
- **Validation:** `Joi` or `Zod` (input sanitization).
- **ORM:** Prisma or TypeORM (Node.js), SQLBoiler (Go).
- **Security:** `Helmet` (HTTP headers), `Argon2` (password hashing).
- **Testing:** Jest/Supertest (Integration), `K6` (Load/Performance testing).

Senior Architect’s Final Note:

To meet the **"Immutable logs"** requirement, I recommend moving beyond simple DB triggers. Implement a **write-only ledger service** that publishes all `TransactionCreated` events to a Kafka topic, which is then persisted into an append-only store (like Amazon QLDB or a dedicated Audit log table with `REVOKE` privileges on `UPDATE/DELETE`). This ensures that even a database admin cannot retroactively alter history.